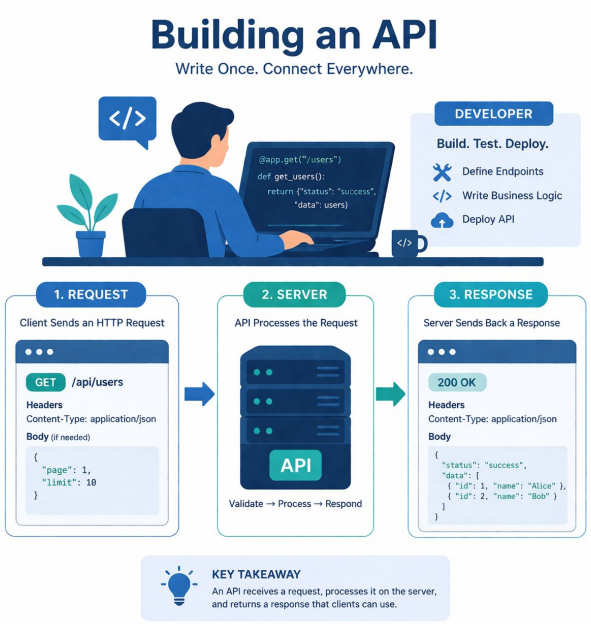
Python Programming Crash

Introduction to Fast API

FastAPI Crash Course — Build Your First API

Learn how to build and test a working API in minutes.

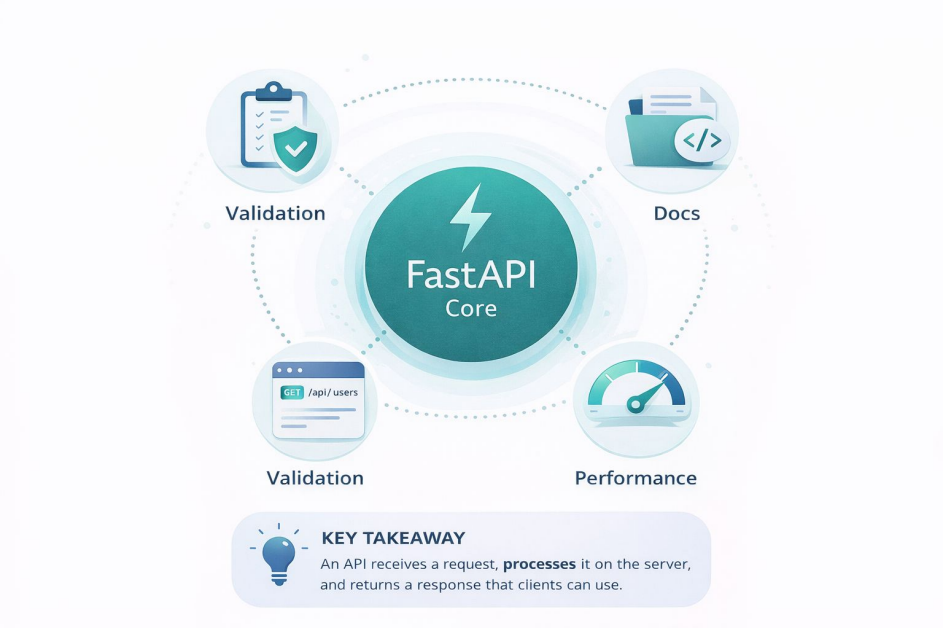
- Understand what FastAPI is
- Build simple API endpoints
- Accept input using JSON
- Test APIs using Swagger UI
- Prepare for the main lab



What is FastAPI?

A modern Python framework for building APIs quickly and efficiently.

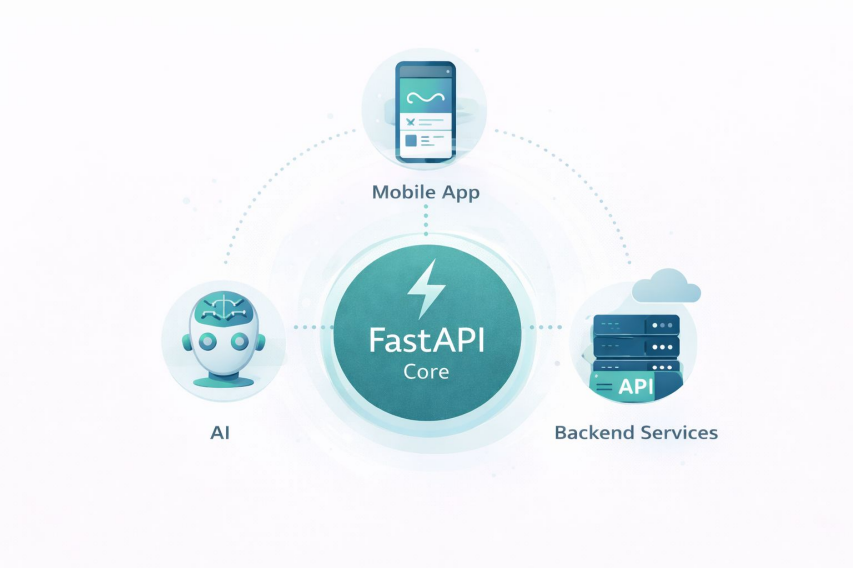
- High performance and lightweight
- Uses Python type hints
- Built-in validation with Pydantic
- Automatic API documentation
- Minimal boilerplate code



Why FastAPI? (Real Use Cases)

FastAPI is widely used for modern backend systems.

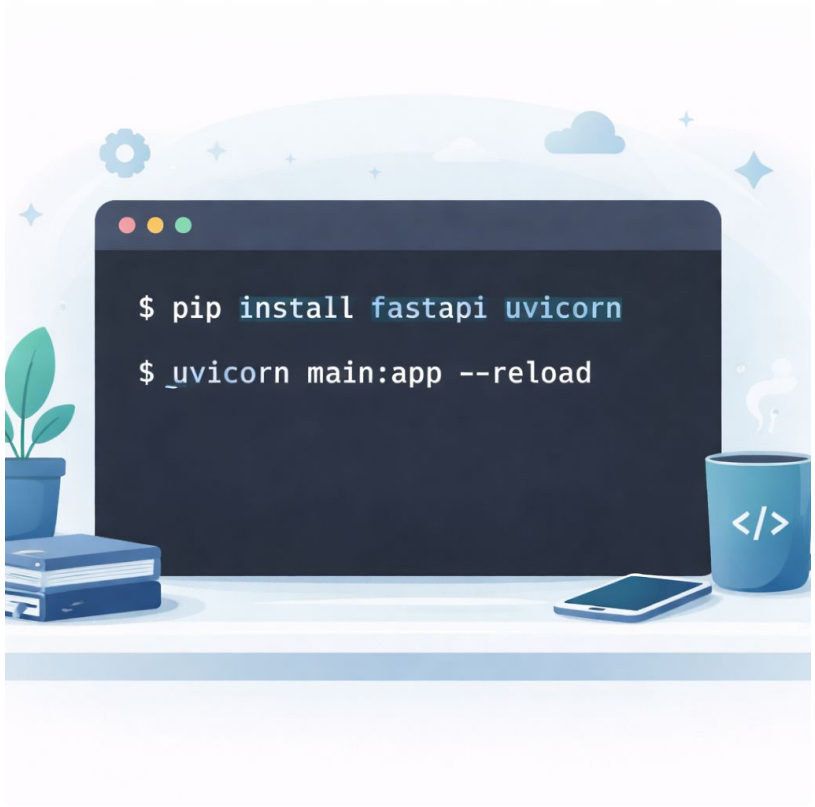
- AI / ML APIs
- Microservices architecture
- Data processing pipelines
- Mobile & web backend services
- Internal enterprise APIs



Setup — Run Your First API

Install and run FastAPI in minutes.

- Install FastAPI and Uvicorn
- Create `main.py`
- Run server locally
- Use auto-reload for development



Your First FastAPI App

Create a minimal working API with just a few lines.

- Create FastAPI instance
- Define a route
- Return JSON response
- No extra configuration needed

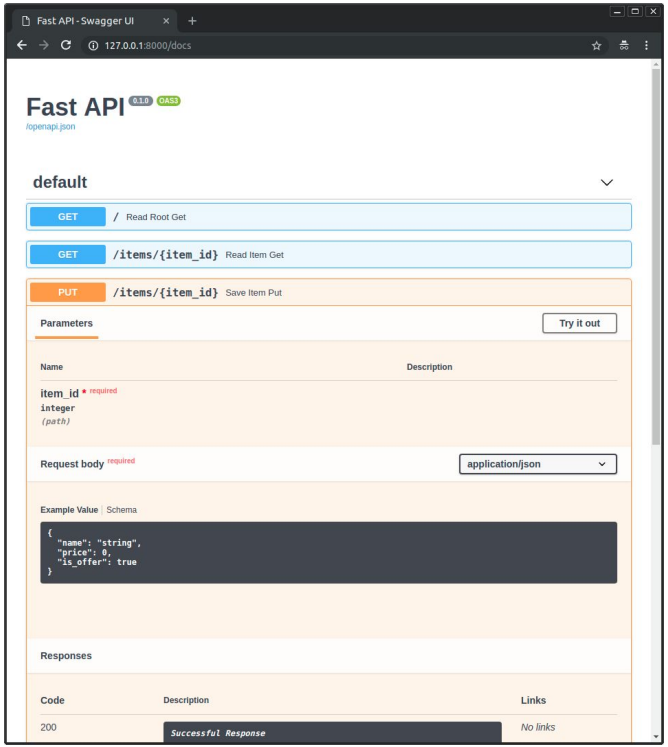
Example code

```
 from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "API is running"}
```

Swagger UI — Test Your API

FastAPI provides built-in interactive API testing.

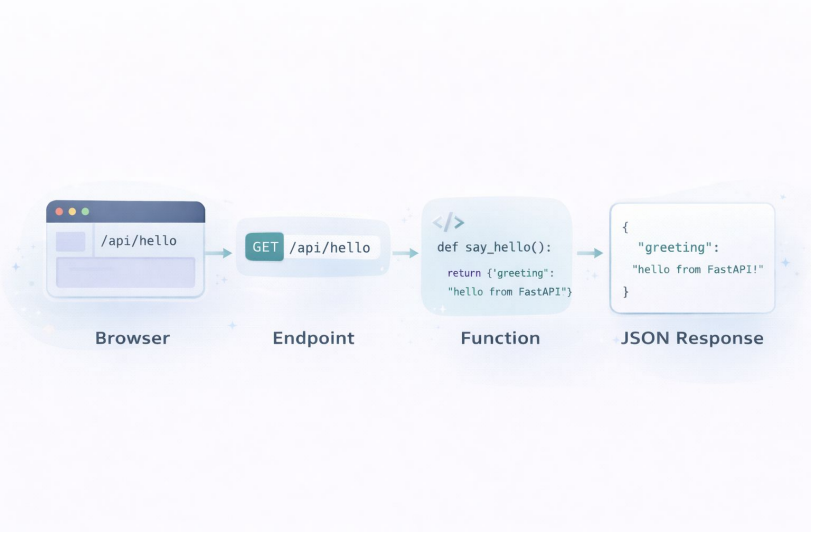
- Access via `/docs`
- No need for Postman
- Auto-generated from code
- Supports live testing



Create Your First Endpoint

Expose a function as an API endpoint.

- Define URL path
- Function contains logic
- Returns JSON response
- Automatically mapped to HTTP



Example Code

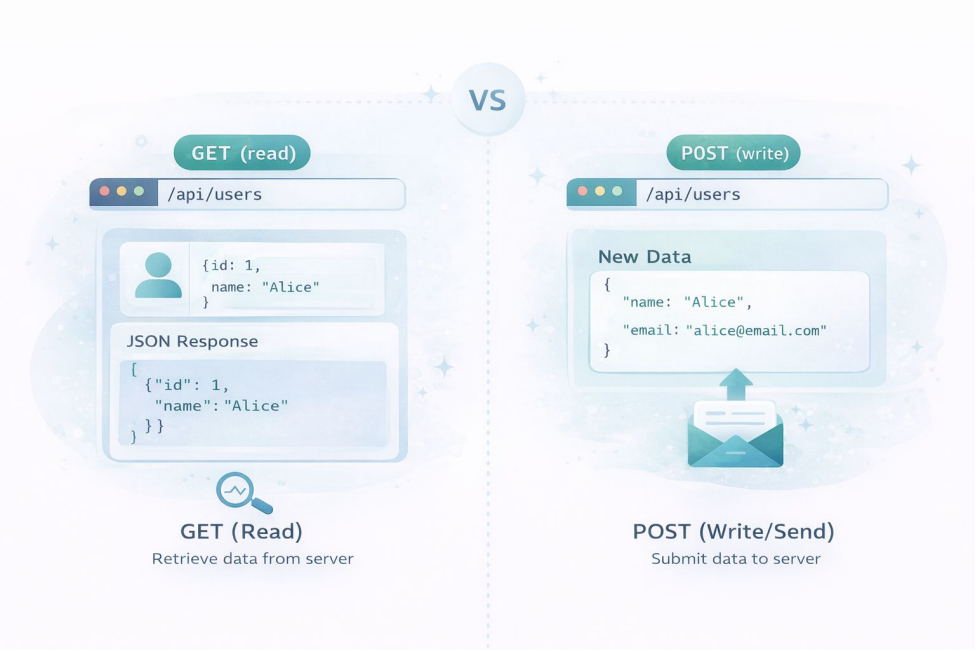
```
@app.get("/hello")
def hello():
    return {"message": "Hello World"}
```

→ Endpoint = function exposed via URL

GET vs POST Requests

Different HTTP methods serve different purposes.

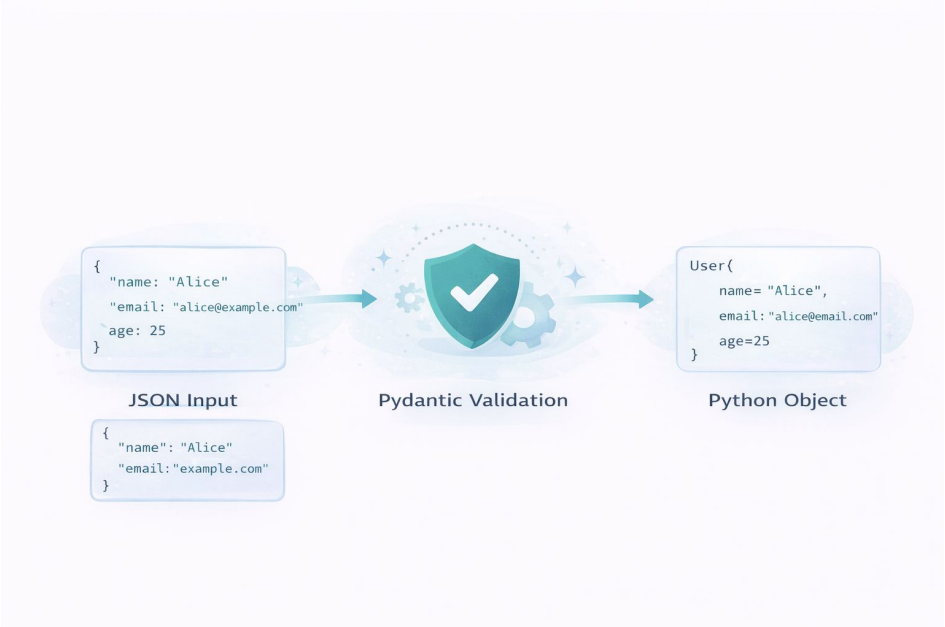
- GET → retrieve data
- POST → send data
- APIs commonly use both
- POST used for processing input
- Important for backend design



Accept Input with Pydantic

Define structured input with automatic validation.

- Use `BaseModel` for input schema
- Define required fields
- Automatic validation
- Cleaner and safer code



Example Code:

```
from pydantic import BaseModel

class UserInput(BaseModel):
    name: str
```

→ Very important for main lab

POST Endpoint with Input

Process user input through an API endpoint.

- Accept JSON input
- Access data via object
- Process input inside function
- Return structured response

Example Code:

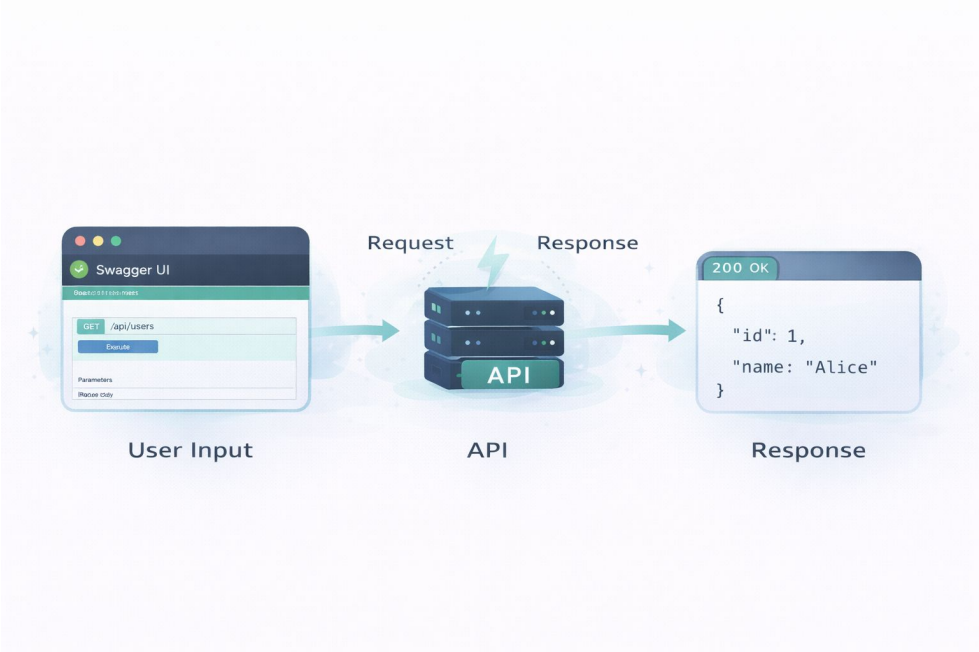
```
 @app.post("/greet")
def greet(data: UserInput):
    return {"message": f"Hello {data.name}"}
```

→ This is same pattern used in your sanitizer API

Test Your API in Swagger

Interactively test endpoints in the browser.

- Open `/docs`
- Select endpoint
- Click “Try it out”
- Send request and view response



Common Mistakes

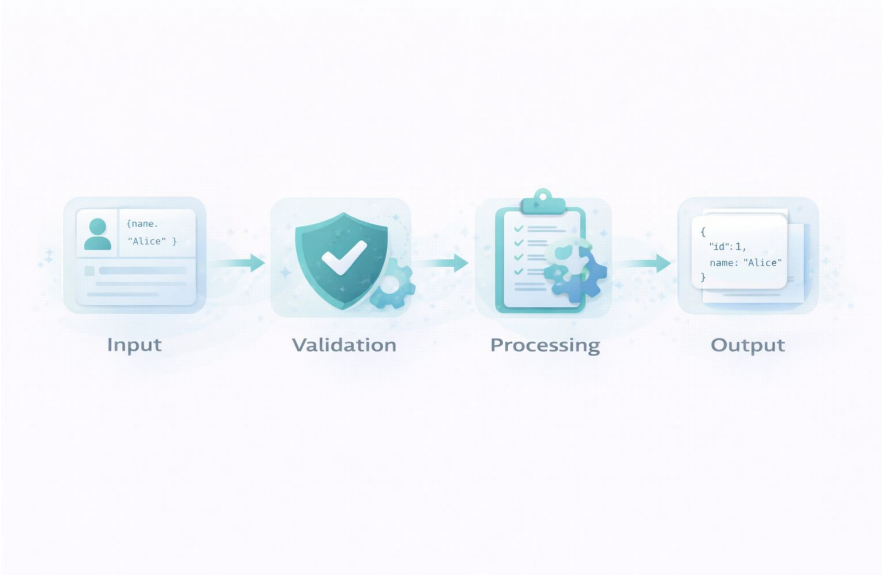
Avoid simple issues when starting with FastAPI.

- Server not running
- Wrong HTTP method
- Invalid JSON format
- Missing required fields
- Typo in endpoint URL

Connecting to Your Main Lab

Apply these basics to a real-world API system.

- Replace simple input with text processing
- Add UTF-8 validation
- Add Unicode normalization
- Add logging and error handling



What’s Next — Smart Text Sanitizer API

Build a real-world API for multilingual text processing.

- Process real input data
- Handle emoji and multilingual text
- Apply validation and normalization
- Build production-style API

API Architecture Overview

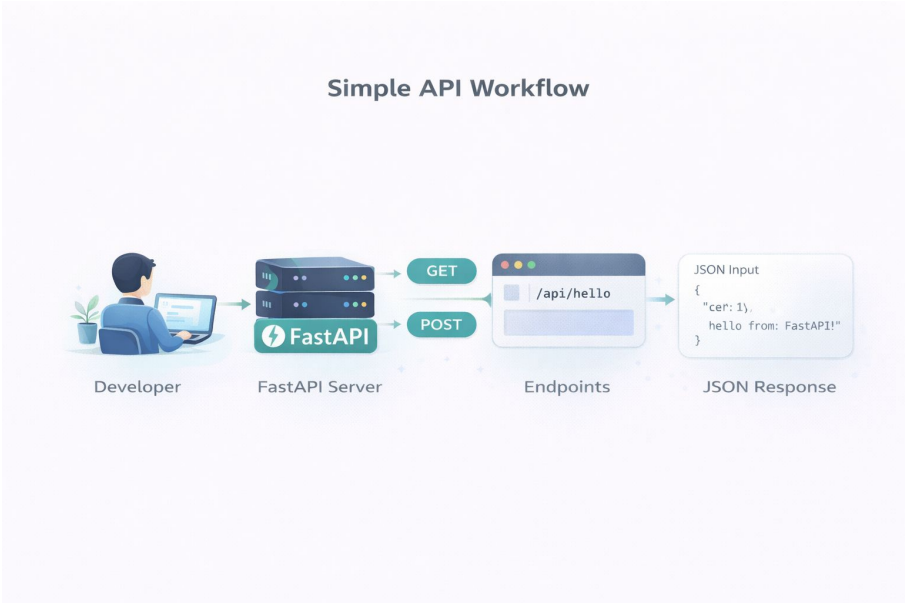
Understand the flow of a real API system.

- Client sends request
- FastAPI receives input
- Validation layer processes data
- Business logic executes
- Response returned to client

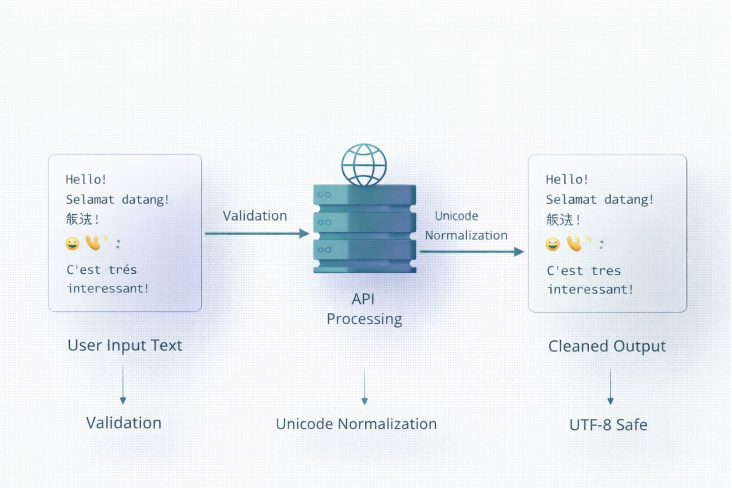
Lab 4 : FastAPI Quick Start (Optional)

By the end of this lab, you will be able to:

- run a FastAPI server locally
- create GET and POST endpoints
- accept JSON input using Pydantic
- test APIs using Swagger UI



Project day 1 : Smart Text Sanitizer API

Build a small FastAPI application that:

- exposes an API endpoint
- accepts text input from a client
- validates text encoding safely
- normalizes Unicode text
- returns cleaned text
- supports emoji and multilingual input
- logs validation and processing errors